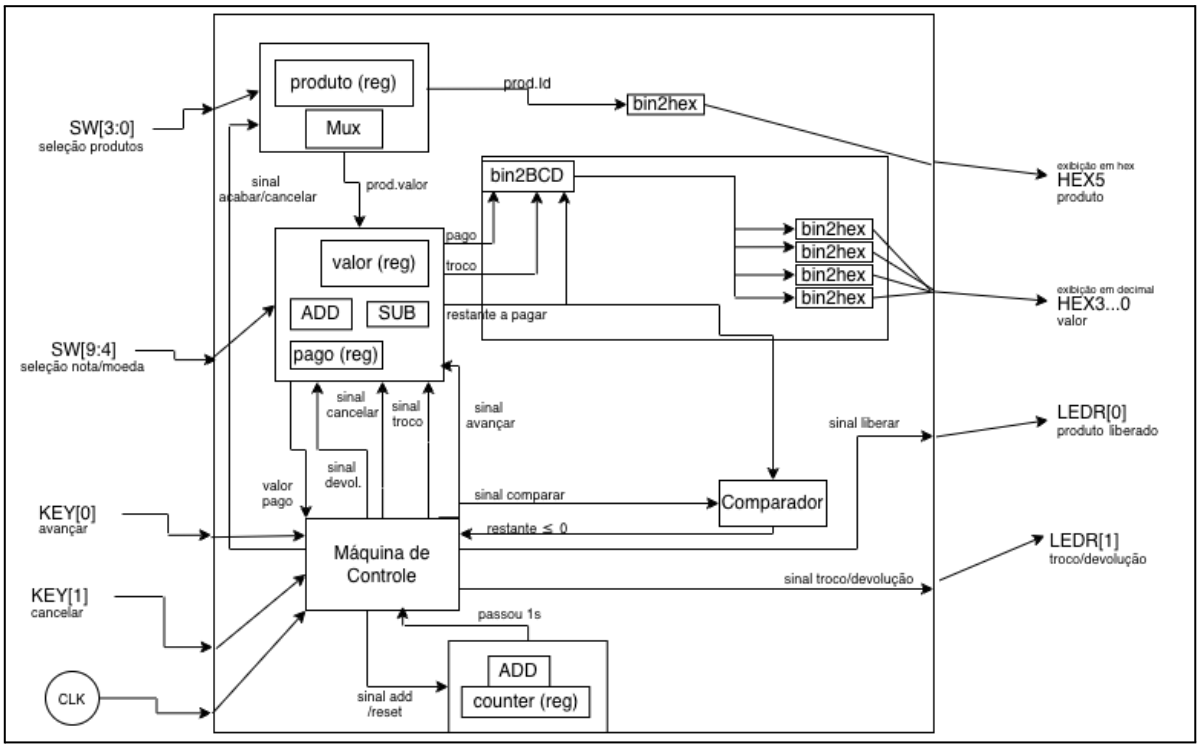
MC613 - P1 - Relatório Final

Caio Lima Albuquerque (288808)
Giovana Jacome Marchetti (177700)
João Pedro Frare (281260)
Victor Yaegashi Setti (206362)

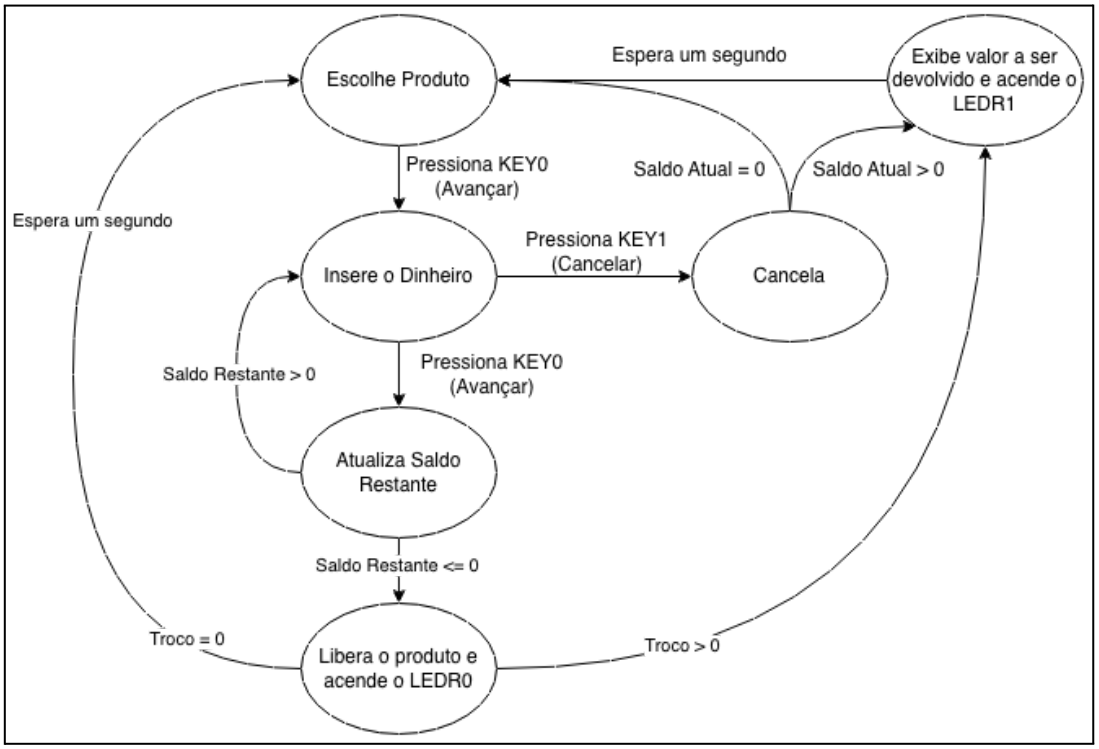
Parte 1 - Planejamento

1.1 Diagramas:

- Diagrama de Blocos



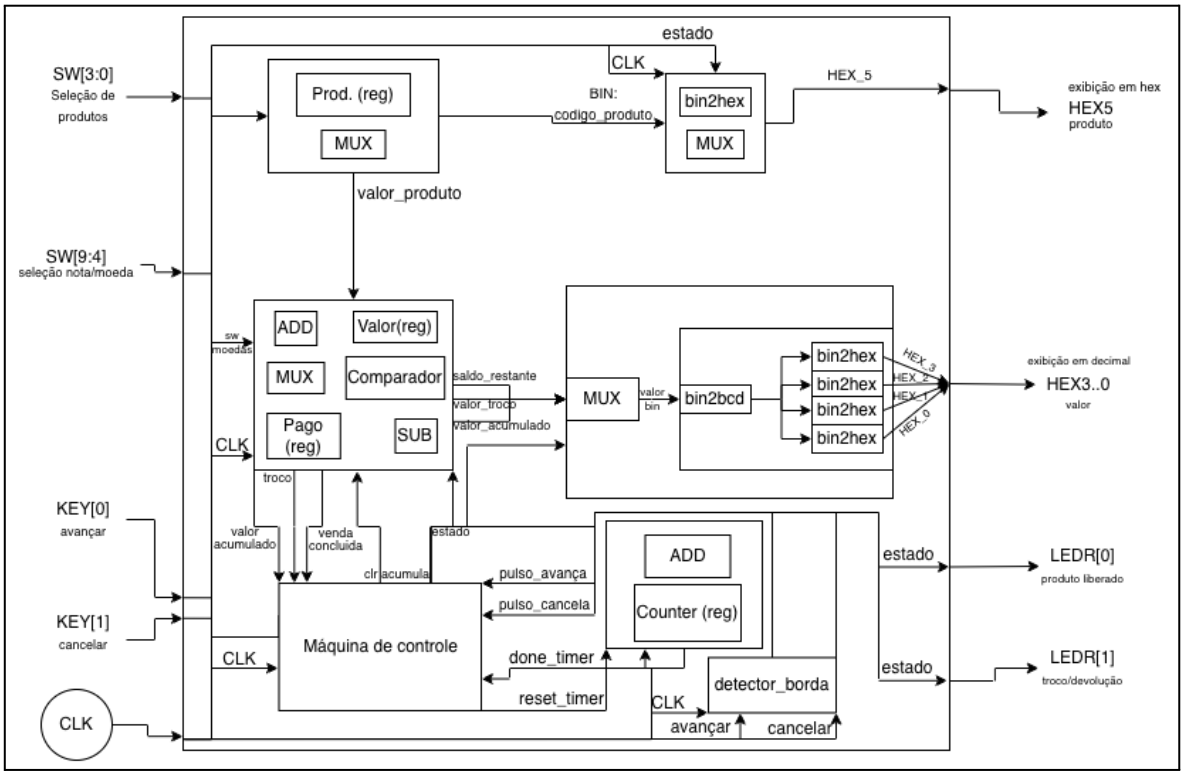
- Diagrama de Estados



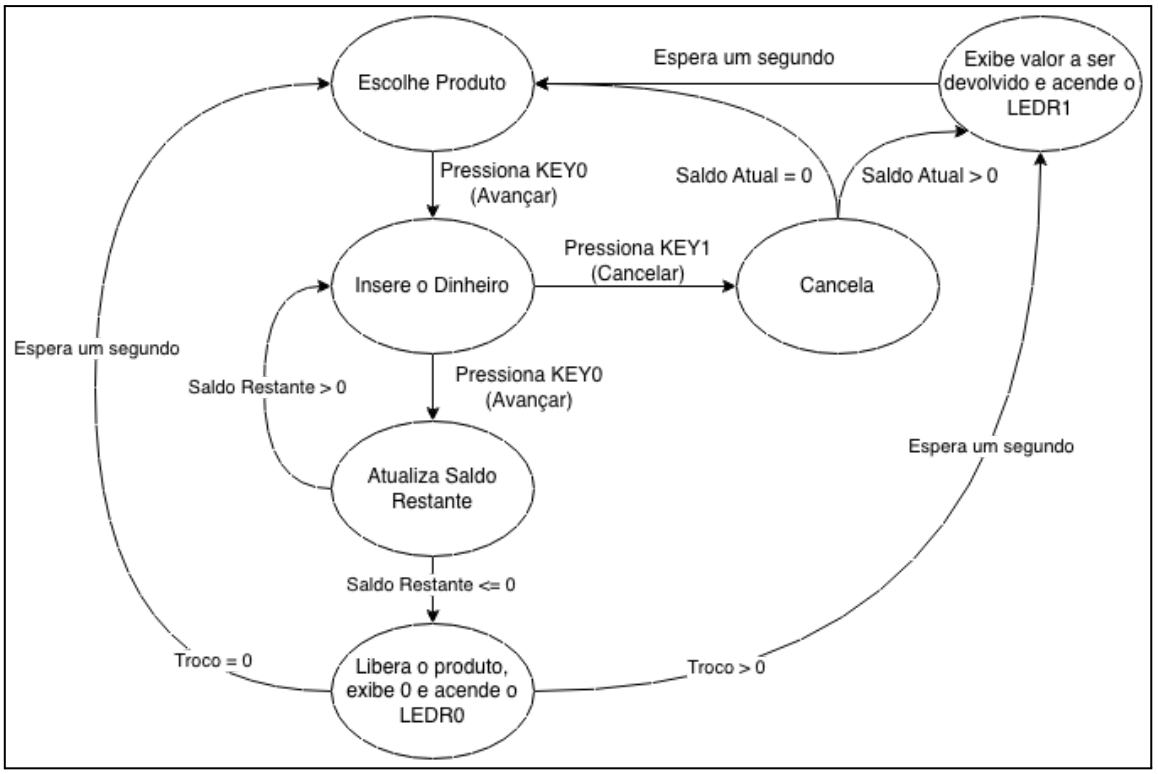
Parte 2 - Entrega Final

2.1 Diagramas:

- Diagrama de Blocos



- Diagrama de Estados



- **Mudanças nos diagramas**
  - **Diagrama de blocos:**
    - Viu-se necessária a inclusão de um módulo “detector_borda” para analisar as entradas assíncronas de avançar e cancelar antes desses sinais chegarem à máquina de estados, pois simplesmente detectar a subida do botão pode encadear em avanços incorretos feitos pela máquina de estados;
    - Foi adicionado um multiplexador para escolher qual dado seria exibido no display de valor em vez de passar os três valores sempre para ele;
    - Adicionou-se um multiplexador no módulo “exibir_produto” para travar o produto escolhido após sair do estado de escolher produto;
    - O comparador foi inserido dentro do módulo da máquina acumuladora pois viu-se desnecessária a existência de um módulo isolado para ele.
  - **Diagrama de estados:**
    - O texto do estado seguinte ao “Atualiza Saldo Restante” foi mudado de “Libera o produto e acende o LEDR[0]” para “Libera o produto, exibe 0 e acende o LEDR[0]” a fim de melhor explicitar o funcionamento do sistema.
    - Foi adicionado mais uma espera de um segundo entre os estados “Libera o produto, exibe 0 e acende o LEDR[0]” e “Exibe valor a ser devolvido e acende o LEDR[1]” pois julgamos ser mais claro e informativo ao usuário se houvesse essa espera entre um estado e outro, possibilitando a leitura e visualização dos LEDs.

2.2 Modularidade:

Nome do módulo: `produto_valor`

Entradas:

- `codigo_produto (STD_LOGIC_VECTOR(3 downto 0))`

Saídas:

- `valor_produto (STD_LOGIC_VECTOR(10 downto 0))`

Descrição da funcionalidade: Conversão do código do produto para seu valor em centavos (binário)

Nome do módulo: `delay_1s`

Entradas:

- `clk (STD_LOGIC)`
- `reset (STD_LOGIC)`

Saídas:

- `done (STD_LOGIC)`

Descrição da funcionalidade: Conta 50 milhões de ciclos de clock (1 segundo) após reset = 1

Nome do módulo: `maquina_acumulador`

Entradas:

- `clk (STD_LOGIC)`
- `clr_acumula (STD_LOGIC)`
- `enable_acumula (STD_LOGIC)`
- `estado_atual (STD_LOGIC_VECTOR(2 downto 0))`
- `valor_produto (STD_LOGIC_VECTOR(10 downto 0))`
- `valor_moeda (STD_LOGIC_VECTOR(10 downto 0))`

Saídas:

- `saldo_restante (STD_LOGIC_VECTOR(10 downto 0))`
- `valor_troco (STD_LOGIC_VECTOR(10 downto 0))`
- `valor_acumulado (STD_LOGIC_VECTOR(10 downto 0))`
- `venda_concluida (STD_LOGIC)`

Descrição da funcionalidade: Realiza o acúmulo do valor inserido pelo usuário e compara com o preço do produto. Calcula o saldo restante e o troco, além de indicar quando a venda foi concluída.

Particularidades de implementação:

- Utiliza um registrador síncrono para acumular valores com base no clock
- O reset (clr_acumula) zera o acumulador
- A lógica de comparação e cálculo de troco é combinacional
- Usa tipo unsigned para operações aritméticas

Nome do módulo: `bin11_to_bcd4`

Entradas:

- `bin (STD_LOGIC_VECTOR(10 downto 0))`

Saídas:

- `bcd (STD_LOGIC_VECTOR(15 downto 0))`

Descrição da funcionalidade: Converte um número binário de 11 bits em sua representação BCD de 4 dígitos (milhar, centena, dezena e unidade).

Particularidades de implementação:

- Implementa o algoritmo Double Dabble (shift-and-add-3)
- Usa variáveis internas (unsigned) dentro de um processo combinacional
- Executa um loop de 11 iterações para realizar a conversão

Nome do módulo: `bin2hex`

Entradas:

- `BIN (STD_LOGIC_VECTOR(3 downto 0))`

Saídas:

- `HEX (STD_LOGIC_VECTOR(6 downto 0))`

Descrição da funcionalidade: Converte um valor binário de 4 bits em sinais para um display de 7 segmentos, permitindo exibir números de 0 a F (hexadecimal).

Nome do módulo: `decode_moedas`

Entradas:

- `sw_moedas (STD_LOGIC_VECTOR(9 downto 4))`

Saídas:

- `valor_moeda (STD_LOGIC_VECTOR(10 downto 0))`
- `moeda_valida (STD_LOGIC)`

Descrição da funcionalidade: Decodifica a entrada das chaves que representam moedas e converte para o valor correspondente em centavos.

Particularidades de implementação:

- Considera válida apenas uma chave ativa por vez
- Caso múltiplas ou nenhuma chave esteja ativa, define moeda como inválida
- Valores são representados em binário (centavos)

Nome do módulo: `maquina_estados`

Entradas:

- `clk (STD_LOGIC)`
- `avancar (STD_LOGIC)`
- `cancelar (STD_LOGIC)`
- `valor_acumulado (STD_LOGIC_VECTOR(10 downto 0))`
- `troco (STD_LOGIC_VECTOR(10 downto 0))`
- `venda_concluida (STD_LOGIC)`
- `done_timer (STD_LOGIC)`

Saídas:

- `reset timer (STD_LOGIC)`
- `estado_out (STD_LOGIC_VECTOR(2 downto 0))`
- `clr_acumula (STD_LOGIC)`

Descrição da funcionalidade: Controla o fluxo principal do sistema por meio de uma máquina de estados finitos, gerenciando as etapas da venda, como seleção do produto, inserção de moedas, liberação do produto e devolução de troco e de cancelamento.

Particularidades de implementação:

- Possui sinal de reset (rst) para retornar ao estado inicial
- Transições de estado dependem de sinais como `avancar`, `moeda_valida` e `venda_concluida`.
- Separação entre lógica de estado (sequencial) e lógica de saída (combinacional)
- Utiliza tipo enumerado para definição dos estados

Nome do módulo: `mux_display_selector`

Entradas:

- `estado (STD_LOGIC_VECTOR(2 downto 0))`
- `saldo_restante (STD_LOGIC_VECTOR(10 downto 0))`
- `valor_troco (STD_LOGIC_VECTOR(10 downto 0))`
- `valor_acumulado (STD_LOGIC_VECTOR(10 downto 0))`

Saídas:

- `saida_bin (STD_LOGIC_VECTOR(10 downto 0))`

Descrição da funcionalidade: Seleciona qual valor será exibido no display (valor acumulado, saldo restante ou troco), com base em um sinal de controle (estado), permitindo multiplexar diferentes valores para visualização.

Particularidades de implementação:

- Implementado como um multiplexador (MUX)
- Seleção baseada em sinal de controle (estado)

Nome do módulo: `detector_borda`

Entradas:

- `clk (STD_LOGIC)`
- `botao_in (STD_LOGIC)`

Saídas:

- `pulso_out (STD_LOGIC)`

Descrição da funcionalidade: Garante que os botões só serão registrados na borda de subida.

Nome do módulo: `exibir_produto`

Entradas:

- `estado (STD_LOGIC_VECTOR(2 downto 0))`
- `clk (STD_LOGIC)`
- `BIN (STD_LOGIC_VECTOR(3 downto 0))`

Saídas:

- `HEX (STD_LOGIC_VECTOR(6 downto 0))`

Descrição da funcionalidade: Define o código do produto à ser exibido no display de 7 segmentos, dependendo do estado.

Particularidades de implementação: Se o estado é “selecionar produto”, o display exibe direto dos switches. Se não, exibe o produto anterior (produto selecionado).

Nome do módulo: `vending_machine`

Entradas:

- `CLOCK_50 (STD_LOGIC)`
- `KEY (STD_LOGIC_VECTOR(3 downto 0))`
- `SW (STD_LOGIC_VECTOR(9 downto 0))`

Saídas:

- `HEX0 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX1 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX2 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX3 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX5 (STD_LOGIC_VECTOR(6 downto 0))`
- `LEDR(STD_LOGIC_VECTOR(9 downto 0))`

Descrição da funcionalidade: Atua como o módulo de nível superior (Top-Level) que integra todos os componentes do sistema de máquina de vendas. Ele gerencia a interface com o usuário (botões e chaves), coordena o fluxo de estados e controla a exibição de informações nos displays de 7 segmentos e LEDs de status.

Particularidades de implementação:

- Instancia múltiplos submódulos, criando a hierarquia do projeto
- Mapeia as chaves SW(3:0) para a seleção do produto e SW(9:4) para a inserção de moedas, enquanto os botões KEY controlam o fluxo de confirmação e cancelamento.
- Converte estados internos em sinais visuais diretos nos LEDs para indicar ao usuário o status da operação (dispensa de produto ou devolução de dinheiro).

Nome do módulo: `hex_display_valor`

Entradas:

- `bin_in (STD_LOGIC_VECTOR(10 downto 0))`

Saídas:

- `HEX_0 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX_1 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX_2 (STD_LOGIC_VECTOR(6 downto 0))`
- `HEX_3 (STD_LOGIC_VECTOR(6 downto 0))`

Descrição da funcionalidade: Recebe um valor em binário do preço do produto, saldo ou troco e retorna os valores à serem exibidos nos displays de sete segmentos. O módulo realiza a conversão da base binária para o formato BCD (Decimal Codificado em Binário) e, em seguida, decodifica cada dígito decimal individualmente para que o valor seja exibido de forma legível (milhar, centena, dezena e unidade).

Particularidades de implementação:

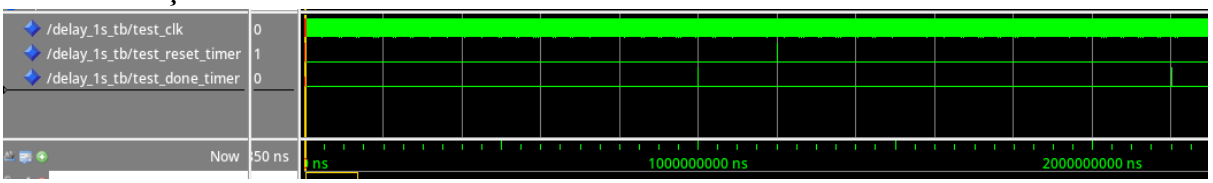
- Instancia o componente bin11_to_bcd4 para transformar o vetor binário original em um formato BCD (Binary Coded Decimal) de 16 bits.
- Instancia quatro cópias do módulo bin2hex, mapeando cada grupo de 4 bits do sinal BCD para um display específico.
- O módulo serve como um encapsulamento que organiza a lógica de conversão e decodificação em uma única unidade.

2.3 Testes:

`produto_valor`: O teste abaixo utiliza todos os códigos de produto possíveis como entrada (segunda coluna), e indica a saída correspondente a entrada dada (binário que representa o valor em centavos do produto correspondente).

```
Iniciando teste
Tempo | Entrada | Saida
10 ns | 0000 | 00001111101
20 ns | 0001 | 00100101100
30 ns | 0010 | 00010101111
40 ns | 0011 | 00111000010
50 ns | 0100 | 00011100001
60 ns | 0101 | 00101011110
70 ns | 0110 | 00011111010
80 ns | 0111 | 00110101001
90 ns | 1000 | 00111110100
100 ns | 1001 | 00101000101
110 ns | 1010 | 01001011000
120 ns | 1011 | 00100010011
130 ns | 1100 | 01010111100
140 ns | 1101 | 00111011011
150 ns | 1110 | 01000001101
160 ns | 1111 | 01100100000
Fim da simulacao
```

`delay_1s`: O testbench abaixo aplica um sinal de clock de 50MHz e pulsos de reset_timer para validar a temporização, esperando que o sinal done_timer seja ativado exatamente após 1s de contagem. O comportamento observado na forma de onda mostra o primeiro done em $1s$ $(10^9 ns)$ ns. Além disso, em $1,2s$ $(1,2.10^9 ns)$, o reset é ativado, o que reinicia a conta e gera o novo sinal done_timer apenas em $2,2s$ $(2,2.10^9 ns)$. O resultado é correto, pois o módulo demonstrou precisão no intervalo de tempo e resposta adequada ao comando de reinicialização.



`hex_display_valor`: O testbench aplica valores automáticos de 0 a 1500 na entrada bin_in, esperando que o módulo decodifique esses valores para os quatro displays de 7 segmentos (HEX_0 a HEX_3). O comportamento observado é registrado via log no console, onde cada mudança de sinal é impressa para validar se os padrões binários de saída correspondem corretamente aos dígitos decimais. O resultado está correto pois os sinais capturados refletiram a conversão de milhar, centena, dezena e unidade para a codificação de 7 segmentos.

```
# Entrada: 000 | SaÃ-da:
# 1000000
# 1000000
# 1000000
# 1000000
# Entrada: 001 | SaÃ-da:
# 1111001
# 1000000
# 1000000
# 1000000
# Entrada: 002 | SaÃ-da:
# 0100100
# 1000000
# 1000000
# 1000000
# Entrada: 003 | SaÃ-da:
# 0110000
# 1000000
# 1000000
# 1000000
# Entrada: 004 | SaÃ-da:
# 0011001
# 1000000
# 1000000
# 1000000
# Entrada: 005 | SaÃ-da:
# 0010010
# 1000000
# 1000000
# 1000000
# Entrada: 006 | SaÃ-da:
# 0000010
# 1000000
# 1000000
# 1000000
# Entrada: 007 | SaÃ-da:
# 1111000
# 1000000
# 1000000
# 1000000
# Entrada: 008 | SaÃ-da:
# 0000000
# 1000000
# 1000000
# 1000000
```

```
# Entrada: 012 | SaÃ-da:
# 0000000
# 1111001
# 1000000
# 1000000
# Entrada: 013 | SaÃ-da:
# 0010000
# 1111001
# 1000000
# 1000000
# Entrada: 014 | SaÃ-da:
# 1000000
# 0100100
# 1000000
# 1000000
# Entrada: 015 | SaÃ-da:
# 1111001
# 0100100
# 1000000
# 1000000
# Entrada: 016 | SaÃ-da:
# 0100100
# 0100100
# 1000000
# 1000000
# Entrada: 017 | SaÃ-da:
# 0110000
# 0100100
# 1000000
# 1000000
# Entrada: 018 | SaÃ-da:
# 0011001
# 0100100
# 1000000
# 1000000
# Entrada: 019 | SaÃ-da:
# 0010010
# 0100100
# 1000000
# 1000000
# Entrada: 01A | SaÃ-da:
# 0000010
# 0100100
# 1000000
# 1000000
```

```
# Entrada: 086 | SaÃ-da:
# 0011001
# 0110000
# 1111001
# 1000000
# Entrada: 087 | SaÃ-da:
# 0010010
# 0110000
# 1111001
# 1000000
# Entrada: 088 | SaÃ-da:
# 0000010
# 0110000
# 1111001
# 1000000
# Entrada: 089 | SaÃ-da:
# 1111000
# 0110000
# 1111001
# 1000000
# Entrada: 08A | SaÃ-da:
# 0000000
# 0110000
# 1111001
# 1000000
# Entrada: 08B | SaÃ-da:
# 0010000
# 0110000
# 1111001
# 1000000
# Entrada: 08C | SaÃ-da:
# 1000000
# 0011001
# 1111001
# 1000000
# Entrada: 08D | SaÃ-da:
# 1111001
# 0011001
# 1111001
# 1000000
# Entrada: 08E | SaÃ-da:
# 0100100
# 0011001
# 1111001
# 1000000
```

```
# Entrada: 5D4 | SaÃ-da:
# 0100100
# 0010000
# 0011001
# 1111001
# Entrada: 5D5 | SaÃ-da:
# 0110000
# 0010000
# 0011001
# 1111001
# Entrada: 5D6 | SaÃ-da:
# 0011001
# 0010000
# 0011001
# 1111001
# Entrada: 5D7 | SaÃ-da:
# 0010010
# 0010000
# 0011001
# 1111001
# Entrada: 5D8 | SaÃ-da:
# 0000010
# 0010000
# 0011001
# 1111001
# Entrada: 5D9 | SaÃ-da:
# 1111000
# 0010000
# 0011001
# 1111001
# Entrada: 5DA | SaÃ-da:
# 0000000
# 0010000
# 0011001
# 1111001
# Entrada: 5DB | SaÃ-da:
# 0010000
# 0010000
# 0011001
# 1111001
# Entrada: 5DC | SaÃ-da:
# 1000000
# 1000000
# 0010010
# 1111001
```

`mux_display_selector`: O testbench aplica diferentes combinações no sinal de seleção estado enquanto mantém valores fixos nas entradas (100, 50 e 25), esperando que a saída saida_bin reflita o valor correto à se exibir. O comportamento observado mostra a seleção correta do saldo no estado "000", do valor acumulado em "011" e do troco em "100".

```
** Note: --- Iniciando Teste do Mux ---
   Time: 0 ps  Iteration: 0  Instance: /mux_display_selector_tb
** Note: Estado 000 (Saldo) -> Saida: 100
   Time: 10 ns  Iteration: 0  Instance: /mux_display_selector_tb
** Note: Estado 011 (Acumulado) -> Saida: 50
   Time: 20 ns  Iteration: 0  Instance: /mux_display_selector_tb
** Note: Estado 100 (Troco) -> Saida: 25
   Time: 30 ns  Iteration: 0  Instance: /mux_display_selector_tb
** Note: Estado Others (Zero) -> Saida: 0
   Time: 40 ns  Iteration: 0  Instance: /mux_display_selector_tb
** Note: --- Fim do Teste ---
   Time: 40 ns  Iteration: 0  Instance: /mux_display_selector_tb
```

`decode_moedas`: Verifica a tradução das chaves físicas em valores decimais. Valida cada moeda individualmente e testa a flag de erro (`moeda_valida`) quando múltiplas chaves são acionadas simultaneamente.

```
# ** Note: --- Iniciando Teste do Decodificador de Moedas ---
#    Time: 0 ps  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 1 (Zero) -> Valida: '0' | Valor: 0
#    Time: 10 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 2 (5c)   -> Valida: '1' | Valor: 5
#    Time: 20 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 3 (10c)  -> Valida: '1' | Valor: 10
#    Time: 30 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 4 (25c)  -> Valida: '1' | Valor: 25
#    Time: 40 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 5 (50c)  -> Valida: '1' | Valor: 50
#    Time: 50 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 6 (1R)   -> Valida: '1' | Valor: 100
#    Time: 60 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 7 (2R)   -> Valida: '1' | Valor: 200
#    Time: 70 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: Caso 8 (Multi)-> Valida: '0' | Valor: 0
#    Time: 80 ns  Iteration: 0  Instance: /decode_moedas_tb
# ** Note: --- Fim do Teste do Decodificador ---
#    Time: 80 ns  Iteration: 0  Instance: /decode_moedas_tb
```

`maquina_acumulador`: O testbench aplica diferentes valores de valor_produto e valor_moeda sob o controle do sinal enable_acumula e do estado_atual, simulando compras exatas, com troco e bloqueios de segurança. O comportamento observado demonstra o acúmulo correto de moedas, o cálculo de saldo restante e a ativação de venda_concluida com o troco correspondente quando o valor inserido excede o preço. O módulo ainda ignorou entradas de moedas em estados inválidos (como no teste 3), garantindo a integridade do sistema.

```
# ==================================================
# INICIANDO SIMULACAO DA MAQUINA DE VENDAS
# ==================================================
# ** Warning: NUMERIC_STD.">=": metavalue detected, returning FALSE
#    Time: 0 ps  Iteration: 0  Instance: /maquina_acumulador_tb/uut
# --- TESTE 1: Compra Exata (Produto 0 = R$ 1,25) ---
# Produto Selecionado! Preco: 125 centavos.
# [!] Inseriu moeda de 100 centavos.
#  -> Valor Acumulado atual: 100
#  -> Saldo Restante a pagar: 25
# [!] Inseriu moeda de 25 centavos.
#  -> Valor Acumulado atual: 125
# [SUCESSO] Venda Concluida! Troco gerado: 0
# --- TESTE 2: Compra com Troco (Produto 1 = R$ 3,00) ---
# Produto Selecionado! Preco travado internamente em: 300 centavos.
# [!] Inseriu moeda de 200 centavos.
#  -> Saldo Restante a pagar: 100
# [!] Inseriu moeda de 200 centavos.
#  -> Valor Acumulado Total: 400
# [SUCESSO] Venda Concluida! Troco a devolver: 100
# --- TESTE 3: Bloqueio de moeda em estado invalido ---
# Mudou para estado diferente de 'Inserir Dinheiro' (010).
# [!] Tentou inserir 50 centavos.
#  -> Valor Acumulado atual (deve continuar o mesmo): 400
# ==================================================
# FIM DA SIMULACAO
# ==================================================
```

`vending_machine`: O testbench valida o sistema completo, simulando a interação do usuário via chaves (SW) e botões (KEY) com clock de 50MHz. O comportamento é monitorado através do status dos displays HEX (preços, saldos e códigos) e dos LEDs (indicadores de liberação de produto e troco). O resultado global foi considerado correto, pois condiz com o comportamento esperado da máquina em cenários normais e críticos.
Os testes específicos realizados foram:

- Teste 1 (Seleção): Varredura de todos os 16 produtos (0 a 15), confirmando se o código e o preço correto aparecem nos displays ao alterar SW(3:0).

```
# TESTE 1: Verificando selecao de produtos via SW(3:0)
# Produto 0 selecionado.
# [STATUS] SW: 0000000000 | LEDR: 0000000000 | HEX5 (Codigo): 1000000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 1111001
# HEX3: 1000000
# Produto 1 selecionado.
# [STATUS] SW: 0000000001 | LEDR: 0000000000 | HEX5 (Codigo): 1111001
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0110000
# HEX3: 1000000
# Produto 2 selecionado.
# [STATUS] SW: 0000000010 | LEDR: 0000000000 | HEX5 (Codigo): 0100100
# HEX0: 0010010
# HEX1: 1111000
# HEX2: 1111001
# HEX3: 1000000
# Produto 3 selecionado.
# [STATUS] SW: 0000000011 | LEDR: 0000000000 | HEX5 (Codigo): 0110000
# HEX0: 1000000
# HEX1: 0010010
# HEX2: 0011001
# HEX3: 1000000
# Produto 4 selecionado.
# [STATUS] SW: 0000000100 | LEDR: 0000000000 | HEX5 (Codigo): 0011001
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 0100100
# HEX3: 1000000
# Produto 5 selecionado.
# [STATUS] SW: 0000000101 | LEDR: 0000000000 | HEX5 (Codigo): 0010010
# HEX0: 1000000
# HEX1: 0010010
# HEX2: 0110000
# HEX3: 1000000
```

```
# Produto 6 selecionado.
# [STATUS] SW: 0000000110 | LEDR: 0000000000 | HEX5 (Codigo): 0000010
# HEX0: 1000000
# HEX1: 0010010
# HEX2: 0100100
# HEX3: 1000000
# Produto 7 selecionado.
# [STATUS] SW: 0000000111 | LEDR: 0000000000 | HEX5 (Codigo): 1111000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 0011001
# HEX3: 1000000
# Produto 8 selecionado.
# [STATUS] SW: 0000001000 | LEDR: 0000000000 | HEX5 (Codigo): 0000000
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0010010
# HEX3: 1000000
# Produto 9 selecionado.
# [STATUS] SW: 0000001001 | LEDR: 0000000000 | HEX5 (Codigo): 0010000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 0110000
# HEX3: 1000000
# Produto 10 selecionado.
# [STATUS] SW: 0000001010 | LEDR: 0000000000 | HEX5 (Codigo): 0001000
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000010
# HEX3: 1000000
# Produto 11 selecionado.
# [STATUS] SW: 0000001011 | LEDR: 0000000000 | HEX5 (Codigo): 0000011
# HEX0: 0010010
# HEX1: 1111000
# HEX2: 0100100
# HEX3: 1000000
# Produto 12 selecionado.
# [STATUS] SW: 0000001100 | LEDR: 0000000000 | HEX5 (Codigo): 1000110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1111000
# HEX3: 1000000
```

```
# Produto 13 selecionado.
# [STATUS] SW: 0000001101 | LEDR: 0000000000 | HEX5 (Codigo): 0100001
# HEX0: 0010010
# HEX1: 1111000
# HEX2: 0011001
# HEX3: 1000000
# Produto 14 selecionado.
# [STATUS] SW: 0000001110 | LEDR: 0000000000 | HEX5 (Codigo): 0000110
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 0010010
# HEX3: 1000000
# Produto 15 selecionado.
# [STATUS] SW: 0000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000000
# HEX3: 1000000
```

- Teste 2 (Cancelamento): Inserção de R$0,50 seguida pelo acionamento do botão de cancelar (KEYR[1]), verificando se a máquina interrompe a venda e devolve o saldo.

```
# TESTE 2: Insercao de moedas e Cancelamento
# Selecionando R$ 0,50 no SW(7)
# KEY(0) pressionada para inserir saldo.
# [STATUS] SW: 0010000011 | LEDR: 0000000000 | HEX5 (Codigo): 0110000
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0011001
# HEX3: 1000000
# Pressionando KEY(1) para CANCELAR...
# [STATUS] SW: 0000000011 | LEDR: 0000000010 | HEX5 (Codigo): 0110000
# HEX0: 1000000
# HEX1: 0010010
# HEX2: 1000000
# HEX3: 1000000
```

- Teste 3 (Venda com Troco): Compra do Produto 0 (125 centavos) inserindo uma nota de R$2,00. Validou-se a ativação sequencial dos LEDs de dispensa de produto e, posteriormente, do LED de troco.

```
# TESTE 3: Venda com Troco (Produto 0 = 125, Inserindo 200)
# Produto 0 selecionado.
# Inserindo nota de R$ 2,00 (SW9)
# [STATUS] SW: 1000000000 | LEDR: 0000000001 | HEX5 (Codigo): 1000000
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# [STATUS] SW: 1000000000 | LEDR: 0000000010 | HEX5 (Codigo): 1000000
# HEX0: 0010010
# HEX1: 1111000
# HEX2: 1000000
# HEX3: 1000000
```

- Teste 4 (Valor Exato): Compra do Produto 1 (300 centavos) através de três inserções sucessivas de R$1,00, confirmando a finalização da venda sem geração de troco.

```
# TESTE 4: Valor Exato (Produto 1 = 300, Inserindo 3x 100)
# Produto 1 selecionado (Custo 3,00)
# Inserindo R$ 1,00 pela 1a vez.
# [STATUS] SW: 0000000001 | LEDR: 0000000000 | HEX5 (Codigo): 1111001
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0100100
# HEX3: 1000000
# Inserindo R$ 1,00 pela 2a vez.
# [STATUS] SW: 0000000001 | LEDR: 0000000000 | HEX5 (Codigo): 1111001
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1111001
# HEX3: 1000000
# Inserindo R$ 1,00 pela 3a vez.
# [STATUS] SW: 0000000001 | LEDR: 0000000001 | HEX5 (Codigo): 1111001
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# Confirmando finalizacao da venda...
# [STATUS] SW: 0000000001 | LEDR: 0000000001 | HEX5 (Codigo): 1111001
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
```

- Teste 5 e 6 (Segurança de Entrada): Tentativas de erro propositais, como inserir duas notas simultâneas ou avançar sem selecionar nenhuma nota. O sistema bloqueou corretamente o acúmulo de saldo inválido.

```
# TESTE 5: Tentativa de inserir DUAS notas ao mesmo tempo
# [STATUS] SW: 0000000000 | LEDR: 0000000000 | HEX5 (Codigo): 1000000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 1111001
# HEX3: 1000000
# Setando SW(9) e SW(8) em '1' (Invalido)
# O saldo no HEX nao deve ter alterado.
# [STATUS] SW: 1100000000 | LEDR: 0000000000 | HEX5 (Codigo): 1000000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 1111001
# HEX3: 1000000
```

```
# TESTE 6: Prosseguir o estado inserir dinheiro sem nenhuma nota selecionada
# [STATUS] SW: 0000000000 | LEDR: 0000000000 | HEX5 (Codigo): 1000000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 1111001
# HEX3: 1000000
# Pressionando AVANCAR sem notas levantadas...
# O saldo restante nao deve ter diminuido.
# [STATUS] SW: 0000000000 | LEDR: 0000000000 | HEX5 (Codigo): 1000000
# HEX0: 0010010
# HEX1: 0100100
# HEX2: 1111001
# HEX3: 1000000
```

- Teste 7 (Proteção de Fluxo): Tentativa de cancelar a operação enquanto o produto já estava sendo liberado. O sistema ignorou o comando, garantindo que o ciclo de entrega e troco fosse concluído.

```
# TESTE 7: Tentativa de cancelamento durante a liberacao
# Inserindo R$ 2,00 para testar a liberacao...
# [STATUS] SW: 0000000000 | LEDR: 0000000001 | HEX5 (Codigo): 1000000
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# Produto esta sendo liberado! Pressionando CANCELAR no meio do processo...
# [STATUS] SW: 0000000000 | LEDR: 0000000001 | HEX5 (Codigo): 1000000
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# A maquina deve ignorar o cancelamento e prosseguir para o troco.
```

- Teste 8 (Overflow): Inserção massiva de notas, com o objetivo de superar o limite de 2047 centavos (11 bits) para observar o comportamento do acumulador em condição de estouro. Foi observado que, como o produto de maior preço é mais barato do que o valor do overflow, a compra foi realizada e o sistema ciclou para um novo estado de escolher produto e depois de inserir dinheiro, o que explica o dinheiro que restou no acumulador antes de encerrar o último ciclo.

```
# TESTE 8: Overflow de dinheiro (> 2047 centavos)
# [STATUS] SW: 0000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000000
# HEX3: 1000000
# Inserindo 12 notas de R$ 2,00 (Total 2400 > limite de 2047)...
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000010
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0011001
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0100100
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000001 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000001 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000001 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
```

```
# [STATUS] SW: 1000001111 | LEDR: 0000000001 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 1000000
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000000
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000000
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0000010
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0011001
# HEX3: 1000000
# [STATUS] SW: 1000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0100100
# HEX3: 1000000
# Verificando o comportamento do acumulador apos o Overflow:
# [STATUS] SW: 0000001111 | LEDR: 0000000000 | HEX5 (Codigo): 0001110
# HEX0: 1000000
# HEX1: 1000000
# HEX2: 0100100
# HEX3: 1000000
# Cancelando para limpar a maquina...
```

- Teste 9 (Protocolo): Tentativa de inserir dinheiro antes da confirmação do produto. O sistema validou apenas a seleção do item, ignorando o valor financeiro prematuro.

```
# TESTE 9: Inserir dinheiro antes da selecao do produto
# [STATUS] SW: 1000000010 | LEDR: 0000000000 | HEX5 (Codigo): 0100100
# HEX0: 0010010
# HEX1: 1111000
# HEX2: 1111001
# HEX3: 1000000
# Chave de R$ 2,00 levantada antes de confirmar o produto.
# Pressionando Confirmar/Avancar...
# O dinheiro nao deve ter sido somado. Apenas o Produto 2 deve ter sido selecionado.
# [STATUS] SW: 1000000010 | LEDR: 0000000000 | HEX5 (Codigo): 0100100
# HEX0: 0010010
# HEX1: 1111000
# HEX2: 1111001
# HEX3: 1000000
```

`maquina_estados`: O testbench aplica valores aos sinais de controle (avancar, cancelar), monitora o tempo via done_timer e fornece feedbacks de saldo e venda, esperando que o estado transite corretamente entre seleção de produto, inserção de dinheiro, dispensa de produto e devolução de troco e devolução de cancelamento. O comportamento observado confirmou as transições esperadas: o sistema retornou à seleção após cancelamentos, avançou para o estado de "devolver_normal" quando havia saldo acumulado e percorreu corretamente os estados de "dispensar" e "devolver_troco" após vendas concluídas.

```
# 1. Esperando escolher_produto
# Entradas -> AV:0 CANC:0 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Sinal Avancar
# Entradas -> AV:1 CANC:0 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Avancou para inserir_dinheiro (estado_out 001)
# Entradas -> AV:0 CANC:0 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:001 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
```

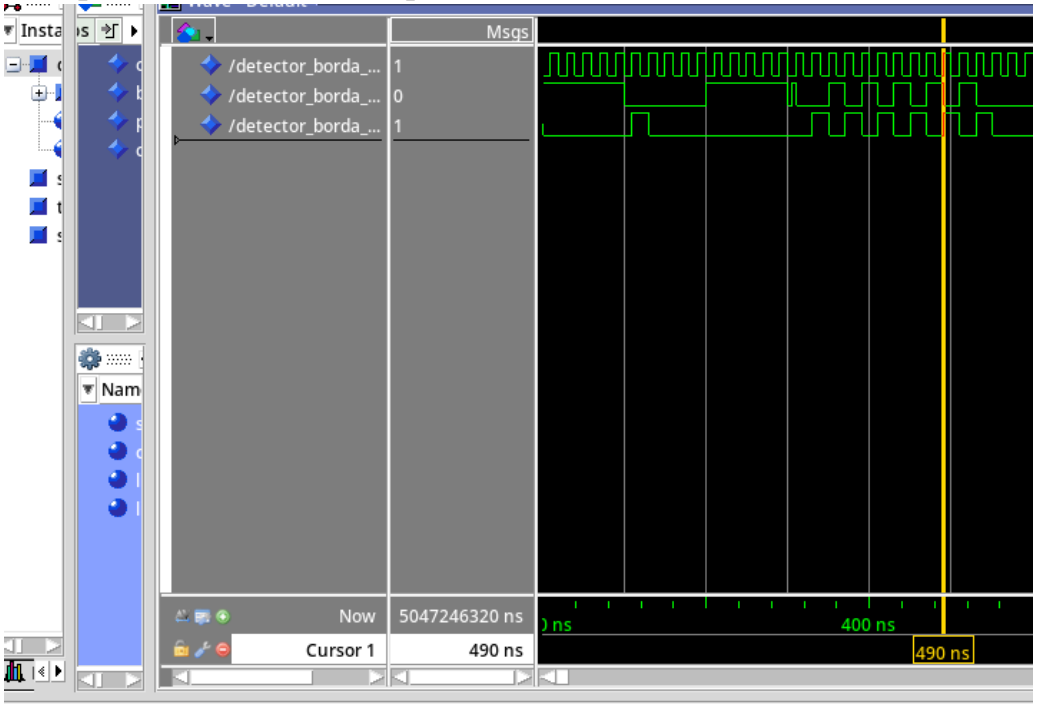
```
# 2. Cancelar sem saldo
# Entradas -> AV:0 CANC:0 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:001 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Sinal cancelar
# Entradas -> AV:0 CANC:1 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:001 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Voltou para escolher_produto (estado_out 000)
# Entradas -> AV:0 CANC:0 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------


# 3. Cancelar com saldo
# Entradas -> AV:0 CANC:0 ACUM:0 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Sinal cancelar
# Entradas -> AV:0 CANC:1 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:001 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Indo para devolver_normal (estado_out 011)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:011 RST_T:0 CLR_ACUM:0
# ---------------------------------------------------
# Dinheiro devolvido. (done_timer=1)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:1
# Saidas   -> ESTADO:011 RST_T:0 CLR_ACUM:1
# ---------------------------------------------------
# Voltou para escolher_produto.
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
```

```
# 4. Venda com troco
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Sinal venda concluida
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:150 VENDA:1 DONE_T:0
# Saidas   -> ESTADO:001 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Indo para dispensar (estado_out 010)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:150 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:010 RST_T:0 CLR_ACUM:0
# ---------------------------------------------------
# Produto dispensado (done_timer=1)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:150 VENDA:0 DONE_T:1
# Saidas   -> ESTADO:010 RST_T:0 CLR_ACUM:0
# ---------------------------------------------------
# Indo para devolver_troco (estado_out 100)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:150 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:100 RST_T:0 CLR_ACUM:0
# ---------------------------------------------------
# Troco devolvido. (done_timer=1)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:150 VENDA:0 DONE_T:1
# Saidas   -> ESTADO:100 RST_T:0 CLR_ACUM:1
# ---------------------------------------------------
# Fim do ciclo. Voltou para escolher_produto
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
```

```
# 5. Venda sem troco
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Sinal venda concluida
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:1 DONE_T:0
# Saidas   -> ESTADO:001 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
# Indo para dispensar (estado_out 010)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:010 RST_T:0 CLR_ACUM:0
# ---------------------------------------------------
# Produto dispensado (done_timer=1)
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:1
# Saidas   -> ESTADO:010 RST_T:0 CLR_ACUM:1
# ---------------------------------------------------
# Indo para escolher_produto
# Entradas -> AV:0 CANC:0 ACUM:500 TROCO:0 VENDA:0 DONE_T:0
# Saidas   -> ESTADO:000 RST_T:1 CLR_ACUM:0
# ---------------------------------------------------
```

`detector_borda`: O testbench submete o componente a uma série de valores no sinal botao_in, variando desde acionamentos normais até condições críticas de temporização, como glitches (ruídos rápidos), condições de corrida e pressões sequenciais. O comportamento esperado é a geração de um único pulso de saída (pulso_out) com duração de exatamente um ciclo de clock para cada transição válida. O comportamento observado confirmou a robustez do circuito, que filtrou corretamente os glitches, tratou condições de corrida e ignorou múltiplos acionamentos sequenciais.




```
# ** Note: --- Iniciando Testes de Estresse ---
#    Time: 0 ps  Iteration: 0  Instance: /detector_borda_tb
# ** Note: Teste 1 (Normal): Verifique se pulso_out deu 1 ciclo de clock.
#    Time: 200 ns  Iteration: 0  Instance: /detector_borda_tb
# ** Note: Teste 2 (Glitch): O pulso NAO deve ser gerado (filtro natural do flip-flop).
#    Time: 305 ns  Iteration: 0  Instance: /detector_borda_tb
# ** Note: Teste 3 (Race Condition): Mudanca junto com o clock. Verifique estabilidade.
#    Time: 330 ns  Iteration: 0  Instance: /detector_borda_tb
# ** Note: Teste 4 (Sequencial): Verifique se foram gerados 5 pulsos distintos.
#    Time: 530 ns  Iteration: 0  Instance: /detector_borda_tb
# ** Note: Teste 5 (Inicializacao): Verifique comportamento no primeiro ciclo.
#    Time: 580 ns  Iteration: 0  Instance: /detector_borda_tb
# ** Note: --- Fim dos Testes  ---
#    Time: 580 ns  Iteration: 0  Instance: /detector_borda_tb
```

`exibir_produto`: O testbench aplica diferentes códigos binários na entrada s_bin e alterna o sinal de controle s_estado, esperando que o display s_hex5 mostre o caractere hexadecimal correspondente apenas quando o estado for "000". O comportamento observado demonstrou que o valor no display é atualizado somente no estado de seleção, mas permanece corretamente travado assim que o estado muda para "001" ou "010", ignorando variações posteriores na entrada.

```
# 1. | Estado: 000 | BIN (Entrada): 0001 | HEX_5 (SaÃ-da): 1111001
# 2. | Estado: 000 | BIN (Entrada): 0101 | HEX_5 (SaÃ-da): 0010010
# 3. | Estado: 001 | BIN (Entrada): 1000 | HEX_5 (SaÃ-da): 0010010
# 4. | Estado: 000 | BIN (Entrada): 1111 | HEX_5 (SaÃ-da): 0001110
# 5. | Estado: 010 | BIN (Entrada): 0000 | HEX_5 (SaÃ-da): 0001110
```

<u>Parte 3 - Finalização</u>

3.1 Estado do Projeto:

O projeto encontra-se **finalizado** e funcionando, uma vez que satisfaz todas as necessidades da máquina de vendas e seus casos de borda, e possui tanto testes em placa quanto em simulador.

3.2 Principais Desafios e Dificuldades:

1. Entender que o top-level é quem liga todos os módulos:
    a. **Identificar:** Conforme fomos desenvolvendo os módulos separadamente, ficamos perdidos sobre como o código de um conseguiria ler uma variável de outro.
    b. **Resolver:** Entendemos que o responsável por este processo é o top-level, após consultar PAD/PED, e conseguimos organizar os sinais de input/output corretamente, bem como ligá-los no top-level.

2. Comunicação em equipe:
    a. **Identificar:** Notamos grande dificuldade em definir como dividiremos as funções e responsabilidades sobre cada módulo entre os membros do grupo. Muito dessa confusão se deu pelo fato de que não estávamos seguros sobre quantos módulos seriam necessários, nem como implementá-los.
    b. **Resolver:** Focamos em analisar os diagramas tanto de Blocos quanto de Estados a fim de estruturarmos melhor os objetivos e funcionamento do sistema, assim conseguimos designar bem as responsabilidades e entender de maneira mais detalhada.

Parte 4 - Apêndice

4.1 Release do Repositório no GitHub:

- MC613 - P1/Release02 - Best Group

OBS: Todos os códigos utilizados estão localizados neste repositório. Favor analisar a Release provida, e não o último commit da main.